
Hybrid Movie Recommendation System

A report submitted to
SVKM's
Narsee Monjee Institute of Management Studies (NMIMS)

**As a part of Semester V
of
Bachelor of Science Applied Mathematical Computing
By**

Juhi Kariya (A005)
Tanushri Shetty (A011)

Under the Guidance of
Dr. Debasmita Mukherjee



**NILKAMAL SCHOOL OF MATHEMATICS,
APPLIED STATISTICS & ANALYTICS**

NMIMS NSOMASA
Ground Floor, SBMP Phase I,
Irla, N. R. G Marg, Opposite Cooper Hospital,
Vile-Parle (West), Mumbai – 400056

November 2024

Certificate

This is to certify that the work described in this thesis entitled “**Hybrid Movie Recommendation System**” has been carried out by Juhi Kariya and Tanushri Shetty under my supervision. I certify that this is their bonafide work. The work described is original and has not been submitted for any degree to this or any other University.

Date: 12th November 2024

Place: Vile Parle, Mumbai

Supervisor:

Dr. Debasmita Mukherjee

Acknowledgement

We take this opportunity to express our heartfelt gratitude towards our mentor, Dr. Debasmita Mukherjee, who supported us throughout the project. She guided us with her advice for topic selection. Dr. Mukherjee was very communicative, which made it easier for us whenever we needed help. She directed us on how we can improve our project. Without her guidance going about this project, would have been difficult.

We would also like to thank Nilkamal School of Mathematics, Applied Statistics & Analytics Mumbai for giving us this opportunity, and other faculties who helped us when we faced obstacles in our project.

Contents

1 Abstract 5

2 Introduction 6

3 Literature Review 7

4 Data Description 8

5 Methodology 9

5.1 Content-Based Filtering 9

5.2 Collaborative Filtering 11

5.2.1 Correlation 11

5.2.2 Singular Value Decomposition 13

5.3 Hybrid Approach 18

6 Results 19

7 Conclusion 21

7.1 Limitations 21

7.2 Future Work: 22

8 References 23

1 Abstract

This project aims to build a recommendation system to suggest movies to a selected user based on a dataset containing 27,278 movies and ratings from 138,493 unique users. The designed recommendation system integrates three techniques — content-based filtering, collaborative filtering using correlation and collaborative filtering using singular value decomposition (SVD).

In the content-based filtering method, we use a TF-IDF matrix and cosine similarity based on the selected user's preferred genres and the most recent top-rated movie to get five recommended movies.

In collaborative filtering we are using two methods: Correlation and Singular Value Decomposition. In correlation method, we identify similar users to the selected user based on the most overlap in watched movies. We sort similar users based on correlation measure and generate weighted average ratings by using the correlation score as weights for all the movies watched by similar users. We filter this list to only contain movies not watched by the selected user and recommend five movies that have the highest weighted average ratings across the entire user base. In SVD method, we use matrix factorisation to predict ratings for all the movies not watched by the selected user and recommend five movies with the highest predicted ratings.

Finally, the hybrid system sorts the fifteen movies from the aforementioned methods based on average ratings from the entire user base. The final movie recommendations contain the two highest rated movies from each method.

The primary objective of this project is to generate a personalised and diverse list of recommendations that take into account different approaches to understanding user preference. To validate the system, we create a user that contains our watched movies and ratings and generate recommendations for ourselves.

2 Introduction

In the age of excess information, everyone is overwhelmed with options and struggles to find movies that appeal to their interests. The primary motivation behind this project is to build a recommendation system that not only helps save time but also provides customised and eclectic movie recommendations.

Despite being useful in some situations, a traditional recommendation system lacks a well-rounded understanding of user preferences. By integrating multiple approaches, the hybrid recommendation system developed for the project hopes to address different aspects of user preference. Genre-specific recommendations are captured by content-based filtering, similar user insights are utilised by collaborative filtering, and underlying patterns in user-movie interactions are taken into account by singular value decomposition.

In addition to the film domain, recommendation systems can be used in the more general domains of music, e-commerce, and online education, where they can boost satisfaction and engagement.

3 Literature Review

The paper titled *A hybrid approach for movie recommendation* by Lekakos and Caravelas [1] proposes a hybrid approach for movie recommendation that combines content-based and collaborative filtering methods. Collaborative filtering is based on looking at users who have similar tastes in movies to find new recommendations, while content-based filtering considers the previously watched movies of the user to predict future preferences. The authors implement the hybrid approach, as well as the pure collaborative and content-based filtering algorithms, in a movie recommendation system called MoRe . The MoRe system collects user ratings, organises them in a user-item matrix, and uses this data along with the features of the movie like genre, cast, and plot keywords to generate recommendations. The hybrid approach is implemented in two versions, called "switching" and "substitute". The performance of the recommendation algorithms is evaluated in terms of predictive accuracy, prediction coverage, and runtime.

The paper titled *Movie Recommendation System* by S. Rajarajeswari, Sharat Naik, Shagun Srikant, M. K. Sai Prakash and Prarthana Uday [2] in addition to the previously described hybrid technique also discusses the use of Singular Value Decomposition (SVD) in the collaborative filtering component of the hybrid movie recommendation system. Their approach utilizes the SVD technique from the Surprise library to predict user preferences. The SVD model is trained on the user ratings data to minimize the RMSE and MAE, allowing the system to estimate the ratings a user would give to a movie they have not yet watched and recommend accordingly.

4 Data Description

We have used the MovieLens 20M Dataset that we found on kaggle [3]. The dataset, as described in Harper and Konstan (2015) [4], includes user-item ratings, timestamps, movie information, and other features that are collected by MovieLens, a movie recommendation service. It contains 2,00,00,263 ratings across 27,278 movies. This data was created by 1,38,493 unique users selected at random. All selected users have rated at least 20 movies.

We utilise the following two datasets to create our database:

- **movie.csv**: Contains information about 27,278 movies.
 - **movieId**: Containing unique movieId for every movie.
 - **title**: Containing movie title.
 - **genres**: Containing genres of each movie.
- **rating.csv**: Contains information about 1,38,493 ratings.
 - **userId**: Containing unique userId for the user who has rated the movie.
 - **movieId**: Containing the movieId of the movie rated.
 - **rating**: Containing the rating between 0.5-5 of the movie rated.
 - **timestamp**: Containing the date and time of when the user rated the movie.

For this project we select ratings from half the users to optimise computing time. We merge both the dataframes along the movieId to create the main database to draw information from for the different recommendation methods.

5 Methodology

In this project we focus on three main approaches to building a recommendation system- content based filtering, collaborative filtering using two methods: correlation and singular value decomposition. In the end we utilise a hybrid approach to integrate results from all the three approaches. The following sections will highlight in detail the idea behind the different approaches and how we have implemented them in this project.

5.1 Content-Based Filtering

Using item features and the user's past behaviour or explicit feedback, content-based filtering suggests items that are similar to the user's past preferences.

Components of content based filtering:

1. Item profile: Consists of the item's feature set. The feature set can be in the form of internal structured characteristics or descriptive metadata.
2. User profile: Consists of the user's past preferences. This can be in the form of items the user has previously shown interest in.

The methodology used in this project employs user specific preferences in the form of recently liked movies and preferred genres. We focus solely on genre to identify similar content. To identify similar movies quantitatively we use TF-IDF and cosine similarity. Our approach is inspired by Kharwal's (2023) [5] use of content-based filtering for fashion products, which gave us valuable insights on how to go about this method for our system.

TF-IDF Matrix:-

The TF-IDF matrix is a numerical representation of how important a term is in the whole database.

- TF Score: Calculates how frequently a term, in this case genre, appears across the 'document' or movie in this instance. Since each genre appears only once per movie, TF will treat each genre as equally present in the movie.
- IDF Score: Calculates how important each term, in this case genre, is for a particular 'document' or movie in this instance. This is calculated by how frequently a genre appears across the entire database. A genre that appears in many movies will have a lower IDF score, indicating that it is common. Similarly, a less frequent genre will have a higher IDF score, as it provides unique information about the particular movie.

Cosine Similarity:-

It is a measure used to calculate similarity between two vectors in a multi-dimensional space. In this project cosine similarity is used to calculate similarity between two movies based on their genres after they have been transformed into vectors using TF-IDF. Cosine similarity measures the angle between two vectors. Mathematically, the cosine similarity between two vectors A and B is defined as:

$$\text{cosine similarity} = \cos \theta = \frac{A.B}{\|A\|.\|B\|}$$

where:

- $A.B$ is the dot product of vectors
- $\|A\|$ and $\|B\|$ are the magnitudes or lengths of vectors A and B
- θ is the angle between the vectors

The cosine similarity score ranges from -1 to 1, where:

- 1 means vectors are identical, meaning perfect similarity
- 0 means vectors are orthogonal, meaning no similarity
- -1 means vectors are completely opposite

How we implemented this method:-

We began by identifying the most recently watched highest rated movie of the selected user. This movie will act as the reference for finding similar movies. We retrieve relevant details of the reference movie like title, movie ID and genre. We then find the selected user's preferred genres by looking at the movies they have rated highly. We filter the database of movies to exclude movies already watched and also movies that are not of the selected user's preferred genres to optimise computation time.

Next we create a content column from the genres of the filtered movies. We apply TF-IDF on the content column. After calculating TF-IDF scores, a matrix is created where each row represents a movie and each column represents a genre. The values in this matrix are the TF-IDF scores, which reflect the relevance of each genre to each movie. This matrix essentially converts the genre information into a numerical form that captures the unique genre profile of each movie. Using the TF-IDF matrix, cosine similarity is calculated between the reference movie and all other movies in the filtered dataset. This calculation produces a similarity score for each movie in the filtered dataset relative to the reference movie. The movies in the filtered set are ranked by their cosine similarity scores, and the top 5 movies with the highest cosine similarity scores are selected as recommendations.

5.2 Collaborative Filtering

Collaborative filtering is a popular recommendation technique that gives suggestions based on collective user data. Unlike content-based methods which rely on item features, collaborative filtering focuses on relationships and preferences across users and items. It relies on the idea that users who have had similar preferences in past are likely to have similar preferences in future as well. By analyzing patterns in user activity history like ratings, clicks or purchases, collaborative filtering can predict items that a user may like based on the behaviour of other users who have similar interests.

It relies on a database of user ratings for various items. These ratings can be explicit, where users directly express their preferences (e.g. on a scale of 1 to 5), or implicit, inferred from user behavior like purchase history, browsing patterns, or watch time.

Collaborative filtering is often divided into two main types:

1. User-based collaborative filtering: Recommends items based on the preferences of similar users. For example, if two users have liked many of the same movies, it is likely that one will enjoy movies the other has rated highly.
2. Item-based collaborative filtering: Recommends items based on similarity between items. For instance, if two movies are frequently watched together, a user who likes one of them is likely to enjoy the other.

The two methods we have followed for implementing collaborative filtering are correlation-based method using Pearson correlation coefficient and matrix factorization method using Singular Value Decomposition.

5.2.1 Correlation

The main concept of correlation method is identifying users that have similar tastes. The system calculates the similarity between users based on their ratings for overlapping items. Here, various similarity measures can be used. We have used Pearson's Correlation Coefficient, which determines whether there is a linear relationship between two users' ratings across the overlapping items. A high positive correlation between two users indicates that they tend to rate items similarly, making them a good prospect for generating recommendations for each other. Once similar users are identified, their ratings are used to predict the target user's preferences for items they haven't yet rated. For example, if several users who share similar tastes with the target user have highly rated a particular movie, the system would recommend that movie to the target user. Thus, using correlation we are performing a user-based collaborative filtering, inspired by the work of Mehmet (2023) [6].

Pearson's Correlation Coefficient:-

The Pearson correlation coefficient between two users x and y is calculated using the following formula:

$$r_{xy} = \frac{\sum (r_{xi} - \bar{r}_x)(r_{yi} - \bar{r}_y)}{\sqrt{\sum (r_{xi} - \bar{r}_x)^2 \sum (r_{yi} - \bar{r}_y)^2}}$$

Where:

- r_{xi} and r_{yi} represent the ratings given by users x and y for item i ,
- $\bar{r}_x$ and $\bar{r}_y$ are the average ratings of users x and y , respectively

The Pearson correlation coefficient r_{xy} ranges from -1 to 1:

- A value of 1 indicates a perfect positive correlation (the users rate items exactly the same)
- A value of 0 indicates no linear correlation (the ratings are independent)
- A value of -1 indicates a perfect negative correlation (the users always rate items oppositely)

How we implemented this method:-

Firstly, to save processing time, we count how many ratings each movie has been given and filter out movies that are rare (i.e. have less than 500 ratings). Doing this, brings down the number of movies from 27,278 to 3,134. These 3,134 movies are labeled as 'common movies'. Then we make a list of the movies watched by the target user that are in the common movies, and display it's ratings in the form of a data frame. We create another data frame that shows how many movies every other user has watched from the target user's movie list. Users who have watched at least 50% of the same movies as the target user are considered to be potential candidates for making recommendations. Once we have these users, we calculate the correlation between the target user and each of the potential user based on their ratings. Users having a correlation greater than 0.5 are labeled as the most similar users.

Then we multiply their correlation score with their ratings for every movie they have watched, to create a weighted rating for each movie. This allows the similar users to have more influence on the final recommendation, helping us generate more relevant recommendations. Once we calculate the weighted ratings, we group the data by movie IDs and calculate the average weighted rating for each movie across all users. This helps us better rank the movies. We filter this list to include only those movies that have an average weighted rating above a certain threshold (in our case it is greater than 3). Finally, we exclude any movies that the target user has already watched and recommend the top five movies with the highest average weighted ratings.

This method provides personalized recommendations based on the preferences of similar users, enabling the system to suggest movies that the target user is more likely to enjoy.

5.2.2 Singular Value Decomposition

Another powerful method is Singular Value Decomposition (SVD). Osmanli (2009) [7] describes how this model doesn't directly compare the users, but using the ratings data it detects latent factors that describe complex relationships between the user preferences and item characteristics. This is useful when we have users that don't have a lot of similarities in their preferences. Once these latent factors are identified, we use the user ratings to predict the target user's preferences for items they haven't yet encountered. So we can say that SVD gives us a mix of user-based and item-based collaborative filtering.

The main element required to use SVD is a user-item matrix. In this matrix, each row represents a particular user and each column represents a particular movie. The value in the matrix are the ratings given by the corresponding user to the corresponding movie. An example can be seen in the below image.

movieId	1	2	5	39	50	150	215	480	539	597
userId										
271.0	1.5	2.5	NaN	NaN	4.0	3.5	NaN	3.0	NaN	2.0
586.0	2.5	3.0	3.0	3.0	4.5	3.0	NaN	3.0	2.0	3.0
614.0	3.0	NaN	NaN	NaN	NaN	3.5	NaN	3.0	NaN	3.0
775.0	4.5	2.0	1.0	3.5	4.0	4.0	2.0	4.5	1.5	2.0
1244.0	4.0	3.5	NaN	3.5	4.0	4.0	NaN	4.5	NaN	NaN

SVD factorizes the user-item matrix into three smaller matrices that capture the latent features of users and items. By reducing the original high-dimensional data into a lower-dimensional form, SVD uncovers hidden patterns that explain user preferences and item characteristics.

Let us understand this mathematically.

SVD takes a rectangular matrix defined as A where A is an $m \times n$ matrix in which the m rows represents the users, and the n columns represents the items. The SVD theorem states:

$$A_{m \times n} = U_{m \times m} S_{m \times n} V_{n \times n}^T \quad (1)$$

Where,

$$U^T U = I_{m \times m}, V^T V = I_{n \times n}$$

So basically SVD takes matrix A as an input and gives us U , S and V^T as an output where,

- the columns of U are the left singular vectors, i.e. it can build back all the columns of A

- V^T has rows that are the right singular vectors, i.e. it can build back all the rows of A
- S (the same dimensions as A) has singular values and is a diagonal matrix

Calculating the SVD consists of finding the eigenvalues and eigenvectors of AA^T and $A^T A$. The orthonormal eigenvectors of $A^T A$ make up the columns of V , the orthonormal eigenvectors of AA^T make up the columns of U . Also, the singular values in S are square roots of eigenvalues from AA^T or $A^T A$ and are the diagonal entries of the S matrix. They are arranged in descending order. If the matrix A is a real matrix, then U and V are also real. The singular values are always real numbers.

Let us try to apply this concept to an example and understand better. Let,

$$A = \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix}$$

In order to find U , we have to first find AA^T . The transpose of A is,

$$A^T = \begin{bmatrix} 3 & -1 \\ 1 & 3 \\ 1 & 1 \end{bmatrix}$$

$$\Rightarrow AA^T = \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix} \begin{bmatrix} 3 & -1 \\ 1 & 3 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 11 & 1 \\ 1 & 11 \end{bmatrix}$$

Then, we have to find the eigenvalues and corresponding eigenvectors of AA^T . We know that eigenvectors are defined by the equation

$$A\vec{v} = \lambda\vec{v}$$

Applying this to AA^T we get our eigenvalues as $\lambda = 10, \lambda = 12$ and their corresponding eigenvectors as $[1 \ -1]$ and $[1 \ 1]$. These eigenvectors are arranged in a matrix based on their eigenvalues. The eigenvector of the largest eigenvalue is placed in column one, the eigenvector of the next largest eigenvalue in column two and so on. So our final matrix is,

$$\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Finally, we have to convert this matrix into an orthogonal matrix which we do by applying the Gram-Schmidt orthonormalization process to the column vectors. The formula to normalize any vector $\vec{p}$ is given by

$$\vec{p}_1 = \frac{\vec{p}}{|\vec{p}|}$$

Applying this formula, we finally get,

$$U = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{bmatrix}$$

We can calculate V in a similar manner. V is based on $A^T A$, which is,

$$A^T A = \begin{bmatrix} 10 & 0 & 2 \\ 0 & 10 & 4 \\ 2 & 4 & 2 \end{bmatrix}$$

We find that the eigenvalues of $A^T A$ come out to be $\lambda = 0, \lambda = 10, \lambda = 12$ and their corresponding eigenvectors are $[1 \ 2 \ -5], [2 \ -1 \ 0], [1 \ 2 \ 1]$. After applying Gram-Schmidt orthonormalization process, our V is given by,

$$V = \begin{bmatrix} \frac{1}{\sqrt{6}} & \frac{2}{\sqrt{5}} & \frac{1}{\sqrt{30}} \\ \frac{2}{\sqrt{6}} & \frac{-1}{\sqrt{5}} & \frac{2}{\sqrt{30}} \\ \frac{1}{\sqrt{6}} & 0 & \frac{-5}{\sqrt{30}} \end{bmatrix}$$

However, we want the transpose of this matrix to find SVD, so,

$$V^T = \begin{bmatrix} \frac{1}{\sqrt{6}} & \frac{2}{\sqrt{6}} & \frac{1}{\sqrt{6}} \\ \frac{2}{\sqrt{5}} & \frac{-1}{\sqrt{5}} & 0 \\ \frac{1}{\sqrt{30}} & \frac{2}{\sqrt{30}} & \frac{-5}{\sqrt{30}} \end{bmatrix}$$

For S , we take the square roots of the non-zero eigenvalues and put them as diagonal entries in S . The largest value is put in s_{11} , the next largest in s_{22} and so on. The non-zero eigenvalues of U and V are always the same, so it doesn't matter which one we take them from.

$$S = \begin{bmatrix} \sqrt{12} & 0 & 0 \\ 0 & \sqrt{10} & 0 \end{bmatrix}$$

Thus we have found all the factorized matrices. We can apply the SVD formula on them and check if we get our original matrix back.

$$\begin{aligned} \therefore A_{m \times n} &= U_{m \times m} S_{m \times n} V_{n \times n}^T \\ &= \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} \sqrt{12} & 0 & 0 \\ 0 & \sqrt{10} & 0 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{6}} & \frac{2}{\sqrt{5}} & \frac{1}{\sqrt{30}} \\ \frac{2}{\sqrt{6}} & \frac{-1}{\sqrt{5}} & \frac{2}{\sqrt{30}} \\ \frac{1}{\sqrt{30}} & \frac{2}{\sqrt{30}} & \frac{-5}{\sqrt{30}} \end{bmatrix} \\ &= \begin{bmatrix} \frac{\sqrt{12}}{\sqrt{2}} & \frac{\sqrt{10}}{\sqrt{2}} & 0 \\ \frac{\sqrt{12}}{\sqrt{2}} & \frac{-\sqrt{10}}{\sqrt{2}} & 0 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{6}} & \frac{2}{\sqrt{6}} & \frac{1}{\sqrt{6}} \\ \frac{2}{\sqrt{5}} & \frac{-1}{\sqrt{5}} & 0 \\ \frac{1}{\sqrt{30}} & \frac{2}{\sqrt{30}} & \frac{-5}{\sqrt{30}} \end{bmatrix} \\ &= \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix} \end{aligned}$$

However, the use of SVD in recommendation systems can be hindered due to sparsity of user-item matrices. This means that there are a lot of missing values in the matrix, which is true for a movie recommendation system as users have not watched all movies in the dataset. SVD requires dense matrices as it decomposes all entries. If we try to apply SVD on a sparse matrix, it will lead to undefined solutions. Hug, N (2017) [8] discusses this in his blog and talks about how Stochastic Gradient Descent (SGD) is often used to overcome this issue.

We have used the Surprise library in python to model this method. It implements SGD in it's core to apply SVD on the user-item matrix. So we are not applying the traditional SVD, but our model is heavily inspired by SVD and is using SGD to handle the sparsity in a matrix.

Performance Metric:-

To evaluate the model's accuracy in predicting ratings, we use Root Mean Square Error (RMSE). The RMSE score provides an insight into how closely the model's predictions match the actual ratings, with a lower RMSE score indicating better performance.

The formula for RMSE is:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

where:

- N is the total number of predictions,
- y_i is the actual rating,
- $\hat{y}_i$ is the predicted rating.

RMSE is always ≥ 0 because it's a square root of squared differences. An RMSE of 0 indicates a perfect fit where predictions match actual values exactly. The upper bound of RMSE is not fixed, as it can increase with larger errors in predictions or if the data itself has larger values. In our case, the ratings range from 0.5 to 5.0, so RMSE will also tend to lie within a range that reflects typical errors within that scale. So, an RMSE near 0.5–1.0 might be reasonable, indicating relatively accurate predictions.

How we implemented this method:-

We began by defining a rating scale ranging from 0.5 to 5.0, which allows for precise rating predictions. The movie data is then loaded and formatted to be compatible with the recommendation library, for easy handling and analysis. Once the data is prepared, it is split into two parts: a training set for building the model and a test set for evaluating its performance, with 20% of the data reserved for testing.

Then, we initialize the SVD model. The model is trained using the training dataset, allowing it to learn patterns in user-item interactions and predict ratings based on

these learned factors. After training, we assess the model's accuracy by predicting ratings for the test data and calculating the Root Mean Square Error (RMSE) as the error metric.

Finally, we define a function to generate personalized movie recommendations for individual users. This function identifies movies the user hasn't rated and predicts potential ratings for these movies. The top five movies are then recommended, based on the highest predicted ratings.

5.3 Hybrid Approach

Hybrid approaches in recommendation system are used to enhance the model performance. By combining multiple techniques, the system can leverage the strengths of each method and reduce their individual drawbacks. Burke [9] classified hybridization techniques into 7 different classes: Weighted, Switching, Mixed, Feature Combination, Cascade, Feature Augmentation, and Meta-Level. Our aim is to offer a diverse set of suggestions to the user, so keeping that in mind we have employed the mixed hybrid approach.

In this approach we gather recommendations from different models and pool them together, allowing the user to consider various kind of suggestions. It's helpful when the system needs to recommend many items at once and can avoid the 'new item' problem, where the new items lack user ratings since content-based filtering can recommend new shows on the basis of their descriptions even if they have not been rated by anyone. However, it does not get around the 'new user' problem, since both the content and collaborative methods need some data about user preferences to start with. We were still inclined to implement this method, since it's one of the easiest hybrid approaches to begin with.

Since each model generates five recommendations, the hybrid approach would give us a list of fifteen movies, which can be overwhelming for users to pick from. So to refine this, we apply the following steps. Firstly, we calculate the average rating for each recommended movie across all users. Then we select the two highest-rated movies from each model's recommendations. Thus, we finally have 6 final movie recommendations to give to the user. This ensures we present only the most highly rated and relevant suggestions, balancing variety with quality.

6 Results

To evaluate the model, we created new users for ourselves where we added our ratings for the movies that we have watched from the database and generate recommendations based on that. User Juhi rated 151 movies and User Tanushri rated 125 movies

Content-Based Filtering

movielid	title	genres
4352	4447 Legally Blonde (2001)	Comedy Romance

Figure 1: User Juhi's Latest Liked Movie Data

Movie: Sabrina (1995), Genre: Comedy|Romance
 Movie: Grumpier Old Men (1995), Genre: Comedy|Romance
 Movie: Waiting to Exhale (1995), Genre: Comedy|Drama|Romance
 Movie: Father of the Bride Part II (1995), Genre: Comedy
 Movie: Mr. Deeds (2002), Genre: Comedy|Romance

Figure 3: User Juhi's Content-Based Recommendation

movielid	title	genres
5872	5971 My Neighbor Totoro (Tonari no Totoro) (1988)	Animation Children Drama Fantasy

Figure 2: User Tanushri's Latest Liked Movie Data

Movie: Stand by Me Doraemon (2014), Genre: Animation|Children|Drama|Fantasy
 Movie: All Dogs Go to Heaven (1989), Genre: Animation|Children|Comedy|Drama|Fantasy
 Movie: Cartoon All-Stars to the Rescue (1990), Genre: Animation|Children|Comedy|Drama|Fantasy
 Movie: Niko & the Way to the Stars Genre: Adventure|Animation|Children|Drama|Fantasy
 Movie: Watership Down (1978), Genre: Adventure|Animation|Children|Drama|Fantasy

Figure 4: User Tanushri's Content-Based Recommendation

As can be observed in the case of both users all the movies recommended via content-based filtering have the same primary genre as their respective latest liked movie.

Collaborative Filtering Using Correlation

After filtering out for rare movies, 62 movies remain for User Juhi and 74 movies remain for User Tanushri.

Movie: To Be or Not to Be (1942), Genre: Comedy|Drama
 Movie: Drop Dead Gorgeous (1999), Genre: Comedy
 Movie: Tekkonkinkreet Genre: Action|Adventure|Animation|Crime|Fantasy
 Movie: Sita Sings the Blues (2008), Genre: Animation|
 Movie: Cheap Detective, The (1978), Genre: Comedy

Figure 5: User Juhi's Collaborative Correlation Recommendation

Movie: Wings of Desire (Himmel über Berlin, Der) (1987), Genre: Drama|Fantasy|Romance
 Movie: Legend of 1900, The Genre: Drama
 Movie: Paper Moon (1973), Genre: Comedy|Crime|Drama
 Movie: Power of One, The (1992), Genre: Drama
 Movie: Love Me If You Dare (Jeux d'enfants) (2003), Genre: Drama|Romance

Figure 6: User Tanushri's Collaborative Correlation Recommendation

In this case it can be observed that the recommended movies are from more diverse genres compared to content-based filtering.

Collaborative Filtering Using SVD

RMSE: 0.7885

Figure 7: User Juhi's RMSE Score for SVD

RMSE: 0.7886

Figure 8: User Tanushri's RMSE Score for SVD

Movie: Good Will Hunting (1997), Genre: Drama|Romance
 Movie: Life Is Beautiful (La Vita è bella) (1997), Genre: Comedy|Drama|Romance|War
 Movie: Playing by Heart (1998), Genre: Drama|Romance
 Movie: Boondock Saints, The (2000), Genre: Action|Crime|Drama|Thriller
 Movie: Crash (2004), Genre: Crime|Drama

Figure 9: User Juhi's SVD based Recommendation

Movie: Shawshank Redemption, The (1994), Genre: Crime|Drama
 Movie: Matrix, The (1999), Genre: Action|Sci-Fi|Thriller
 Movie: Snatch (2000), Genre: Comedy|Crime|Thriller
 Movie: Life Is Beautiful (La Vita è bella) (1997), Genre: Comedy|Drama|Romance|War
 Movie: Godfather, The (1972), Genre: Crime|Drama

Figure 10: User Tanushri's SVD based Recommendation

We can see that for both users we achieved a moderately good RMSE Score, so our model was able to successfully predict ratings for our users.

In this case it can be observed that the recommended movies are from a diverse range of genres and are also well known and highly rated. As an individual method this creates the most comprehensive list of recommendations.

Hybrid Approach

Movie: Grumpier Old Men (1995), Genre: Comedy|Romance
 Movie: Sabrina (1995), Genre: Comedy|Romance
 Movie: Good Will Hunting (1997), Genre: Drama|Romance
 Movie: To Be or Not to Be (1942), Genre: Comedy|Drama|War
 Movie: Life Is Beautiful (La Vita è bella) (1997), Genre: Comedy|Drama|Romance|War
 Movie: Tekkonkinkreet Genre: Action|Adventure|Animation|Crime|Fantasy

Figure 11: User Juhi's Final Recommendations

Movie: Stand by Me Doraemon (2014), Genre: Animation|Children|Drama|Fantasy
 Movie: Shawshank Redemption, The (1994), Genre: Crime|Drama
 Movie: Godfather, The (1972), Genre: Crime|Drama
 Movie: Matrix, The (1999), Genre: Action|Sci-Fi|Thriller
 Movie: Life Is Beautiful (La Vita è bella) (1997), Genre: Comedy|Drama|Romance|War
 Movie: Snatch (2000), Genre: Comedy|Crime|Thriller
 Movie: Wings of Desire (Himmel über Berlin, Der) (1987), Genre: Drama|Fantasy|Romance

Figure 12: User Tanushri's Final Recommendations

Our final list combines all the different approaches and attempts to produce the most comprehensive, personalised and well-rounded list of recommendations.

All relevant codes, inputs and outputs can be found in the project repository [10].

7 Conclusion

By using a mixed hybrid approach, we were successfully able to provide personalized recommendations to a user by making use of content-based filtering as well as collaborative filtering. The content based filtering made sure that user's genre preferences were taken into consideration while suggesting movies. The correlation method in collaborative filtering helped us capture any direct relationship between users and items based on their rating behaviours, while singular value decomposition were useful in capturing complex patterns in user preferences, that correlation might have missed. This allowed us to make more nuanced and diverse predictions for our users. Using a hybrid approach helped us mitigate limitations caused by one method, and utilize the strengths of each method. To test the model's accuracy we also applied this system on our rating history and found some good satisfying recommendations for ourselves.

This fascinating journey allowed us to see concepts from linear algebra and statistics come to life in practical application of recommendation systems. It solidified the idea that mathematics is more than just theory, that it is in fact the driving force behind influential technologies.

7.1 Limitations

Despite the effectiveness of our model, we identified several limitations and challenges while working on it and testing it.

- **Cold Start:** This refers to the "new user" and the "new item" problem. Newly released movies provide difficulty in collaborative filtering because we don't have enough history of ratings for comparison. New users also face issues in getting accurate recommendations as the system doesn't have a user history to make recommendations on. The content-based method partially helps us mitigate this issue, but does not completely solve it.
- **Sparsity Problem:** Collaborative filtering systems rely on overlap between user preferences. So sparsity can create problems if there are only a few users who have rated the same movies, which is very common in a movie database.
- **Limited Features:** In content-based filtering we are only looking at the genres of the movies. Genres generally have very broad categories. For example a horror movie might range from psychological horror to supernatural thrillers. A user may enjoy one subtype but not all. So overall it lacks specificity. In addition to that, there are many other features of a movie that a user might enjoy like pacing, direction, cast, etc., which our dataset does not have.
- **Gray Sheep Problem:** Due to the fact that we halved our ratings dataset to save computation time, for certain target users we were unable to find a good number of similar users because their interests were too niche or unique. So

we had to keep re-running our model until we were able to find more users whose tastes aligned well enough with our target user.

- **Popularity Bias:** For correlation method we had to filter out movies that had fewer ratings because this method tends to favour movies with higher ratings and more user interactions over the less popular ones that might have better matched a user's unique taste. This somewhat lead to repeat recommendations for different users as well as reduced diversity in suggestions.

7.2 Future Work:

While our system gave us promising results, there are several areas where we could improve and enhance its performance and user experience. Below are some potential directions for further development:

- **Improve Hybrid Approach:** We could implement a combination of weighted and switching hybrid approach. The weighted approach would provide weights to each method based on the context. For example, right after a user watches a movie, content-based filtering could be given more weight, while collaborative filtering could be given a higher weight for general movie suggestions. Switching hybrid approach would allow us to switch between our recommendation methods based on their performance. For instance, if one method is under performing because of lack of data, the system must automatically switch to the more appropriate method instead of not giving an output.
- **Expand our content features:** We could enhance our content-based filtering by including more movie features other than genres. Some of these features could be directors, cast, pacing, movie summaries, release years, user reviews, etc. This would help us capture more specific preferences of a user. This also helps collaborative filtering, especially SVD, to figure out more complex relationships between users and items.
- **Alternative Correlation Metrics:** We could explore other correlation coefficients or even explore advanced similarity measures to see if they provide more accurate results.
- **User-friendly Interface:** Making a user-friendly interface would allow our system to be ready for a practical use by letting the users rate movies easily and pick their preferred genres based on their current mood and receive more tailored recommendations. This also increases user satisfaction since it is easier to work with.

8 References

- [1] G. Lekakos, & P. Caravelas, "A hybrid approach for movie recommendation," *Multimedia Tools and Applications*, vol. 36, no. 1-2, pp. 55–70, 2006.
doi:10.1007/s11042-006-0082-7
- [2] S. Rajarajeswari, S. Naik, S. Srikant, M. K. Sai Prakash, & P. Uday, "Movie recommendation system," in *Springer Theses*, pp. 329–340, 2019.
doi:10.1007/978-981-13-5953-8_28
- [3] GroupLens and E. Kim, "MovieLens 20M dataset," Kaggle, 2018. [Online].
Available at: <https://www.kaggle.com/datasets/grouplens/movielens-20m-dataset>
- [4] F. M. Harper, & J. A. Konstan, "The MovieLens datasets," *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, pp. 1–19, 2015.
doi:10.1145/2827872
- [5] A. Kharwal, "Hybrid recommendation system using Python," *The Clever Programmer*, 2023. [Online].
Available at: <https://thecleverprogrammer.com/2023/06/05/hybrid-recommendation-system-using-python/>
- [6] M. ISIK, "Hybrid recommender system," Kaggle, 2023. [Online].
Available at: <https://www.kaggle.com/code/mehmetisik/hybrid-recommender-system>
- [7] O. N. Osmanli, "A singular value decomposition approach for recommendation systems," M.S. thesis, *Middle East Technical University*, 2010.
- [8] N. Hug, "Understanding matrix factorization for recommendation (part 3): SVD for recommendation," 2020. [Online].
Available at: https://nicolas-hug.com/blog/matrix_facto_3
- [9] R. Burke, "User modeling and user-adapted interaction," *User Modeling and User-Adapted Interaction*, vol. 12, no. 4, pp. 331–370, 2002.
doi:10.1023/a:1021240730564
- [10] "Juhi & Tanushri - RD Project Sem 5," *Google Drive*, [Online].
Available at: <https://shorturl.at/zW2vm>